

ERROR HANDLING USING EXCEPTION HANDLING

Aim: To write and execute a Python program to handle runtime errors using exception handling.

Interactive Development Environment (IDE) Used : Jupyter NoteBook

Programming Language Used : Python

Python Concepts used:

Exception handling is a mechanism used to detect and handle runtime errors without abruptly terminating a program. Python uses the try, except, else and finally blocks for exception handling. Built-in exceptions such as ValueError can be handled directly, while a programmer can define a custom exception by creating a class derived from Exception. The raise statement is used to explicitly generate an exception when a specified condition occurs.

Exception Handling Concept	Example Code	Description
try block	<pre>try: statements</pre>	Contains statements that may produce an exception.
except block	<pre>except ValueError: statements</pre>	Handles a particular exception when it occurs.
Multiple except blocks	<pre>except ValueError: ... except TypeError: ...</pre>	Allows different types of exceptions to be handled separately.
Exception object	<pre>except Exception as e:</pre>	Stores exception information in a variable such as e.
Custom Exception	<pre>class MyError(Exception): pass</pre>	Creates a user-defined exception by inheriting from Exception.
raise	<pre>raise MyError("Invalid data")</pre>	Explicitly generates an exception when a condition is not acceptable.
else block	<pre>else: statements</pre>	Executes only when no exception occurs in the try block.
finally block	<pre>finally: statements</pre>	Executes whether an exception occurs or not; commonly used for cleanup.
Built-in Exception	<pre>ValueError</pre>	Occurs when a value has the correct type but an inappropriate value or format.
Exception Message	<pre>print("Error:", e)</pre>	Displays the message associated with the exception.

Python Program:

```
# Experiment 7: Machine Temperature Monitoring

class TemperatureError(Exception):
    """Custom exception for unsafe machine temperature."""
    pass

try:
    temperature = float(input("Enter machine temperature (°C): "))

    if temperature < 0 or temperature > 100:
        raise TemperatureError("Machine temperature is outside the safe range!")
except ValueError:
    print("Error: Please enter a valid numerical temperature.")

except TemperatureError as e:
    print("Temperature Error:", e)
else:
    print("Machine temperature is safe.")
    print("Temperature:", temperature, "°C")

finally:
    print("Temperature checking completed.")
```

Sample Input and Output :

Case 1 – Valid input:

```
Enter machine temperature (°C): 75
Machine temperature is safe.
Temperature: 75.0 °C
Temperature checking completed.
```

Case 2 – Unsafe temperature:

```
Enter machine temperature (°C): 120
Temperature Error: Machine temperature is outside the safe range!
Temperature checking completed.
```

Case 3 – Invalid input:

```
Enter machine temperature (°C): abc
Error: Please enter a valid numerical temperature.
Temperature checking completed.
```

Result :

The Python program was written and executed successfully to demonstrate error handling using built-in and user-defined exceptions.